

REPORT

Course Code: CSE 4408

Course Name: System Analysis and Design

Lab Number: 8

Project Title: BashayJabo

Group: Les Misérables

Group Members:

1. Ridita Alam (ID 220041110)
2. Rahatut Tahrim Mounota (ID 220041122)
3. Saba Atharique (ID 220041130)

1. Initial Logical DFDs

External Entities:

- Student – Commuting IUTians using the system to create or join existing trips for effective ride-sharing.
- External Ride Booking Services – Integrated APIs for booking rides such as Uber/Pathao.
- Payment Client – Online payment APIs such as Bkash/Nagad for fast and secure payment options.
- Credential Database – User login and profile information to save account data.

Major Data Flows:

- Request Ride / Confirmation – System receives requests and relevant information from students to create or join rides, sends confirmation and updates to users.
- Payment Info / Payment Status – Online payment APIs receive trip payment information and send confirmation upon completion.
- Login Request / Authentication – User credentials are read and verified for existing accounts, new credentials added for new users.

Core Processes:

- Authenticate User – User login.
- Create Ride – New ride listing with details and preferences.
- Join Ride – Request to join existing ride.
- Confirm Ride – Approved join requests, status updates on rides.
- Initiate Payment – Fare splitting and online payment integration.
- Communicate with External Services – Ride booking services.

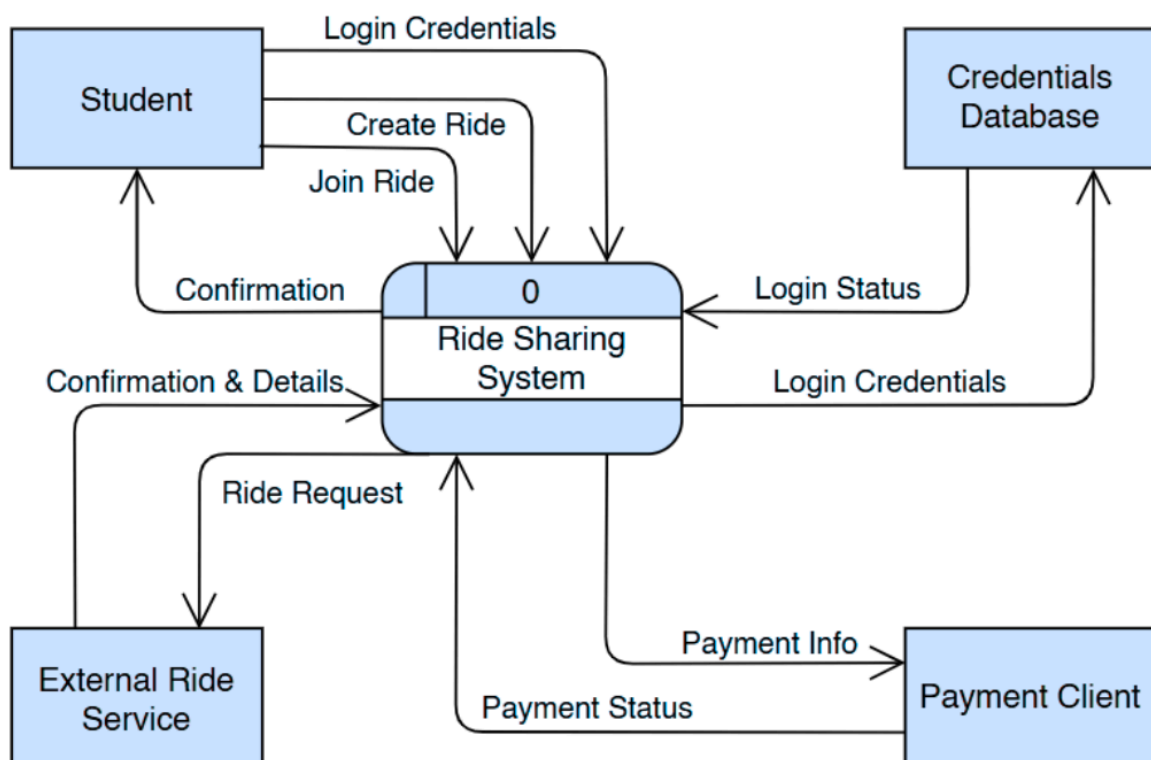
Foundational Data Stores:

- User Data – Profile details, history, login credentials.
- Ride Listings – All created ride offers, past or currently active.
- Ride Requests – Join requests and statuses for created rides.

Context Diagram (Level 0)

The Level 0 Context Diagram gives an overview of the data flow between the system and the four main external entities, Student, Credential Database, External Ride Services, and Payment Client.

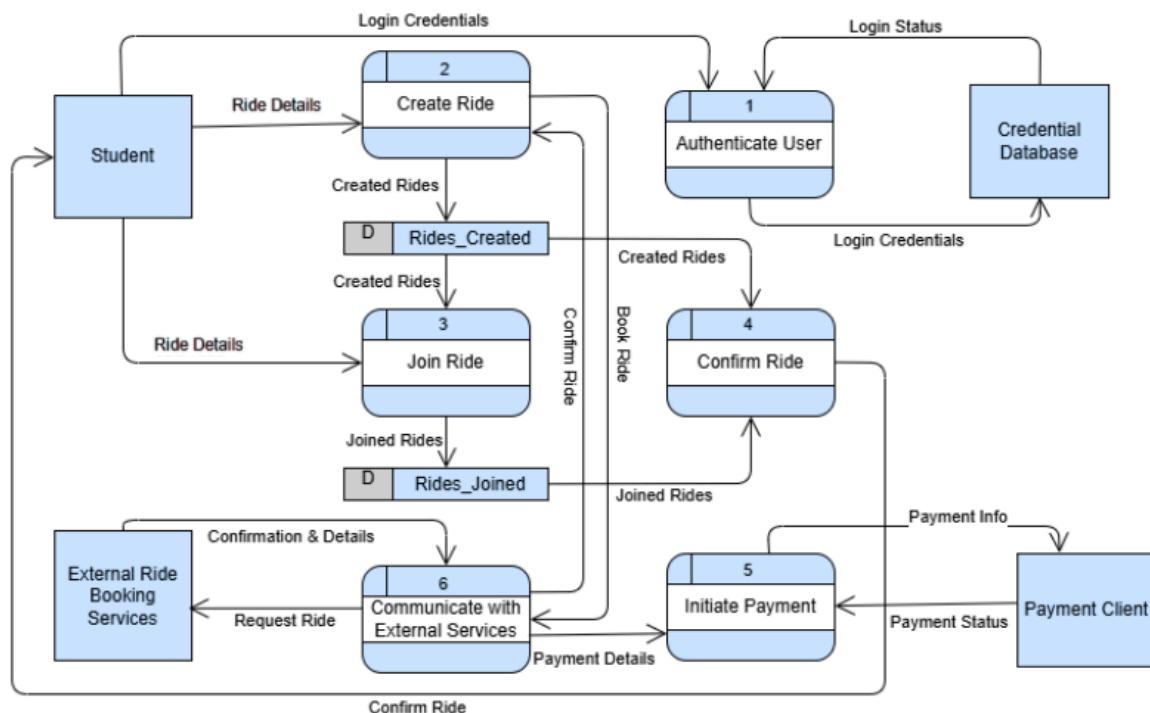
1. A **Student** can request to login, create, or join a ride, providing appropriate information for each action. Upon successful completion of these requests, a confirmation is sent from the system.
2. During user login, the **Credential Database** receives user data such as account email and password, and returns a login status depending on the accuracy of input.
3. While creating rides, the **External Ride Service** may be called if the student is travelling via Uber/Pathao, etc.. A ride booking request is processed and details are sent to the system.
4. At the end of the trip, if the user chooses to pay via online services such as Bkash, the **Payment Client** processes the payment information and sends back a status (failed, pending, completed).



Logical Diagram 0

From the Context Level DFD, the Ride Sharing System has been exploded into 6 core processes, with two supporting data stores. This gives a clearer view into the subprocesses and data flows of the system.

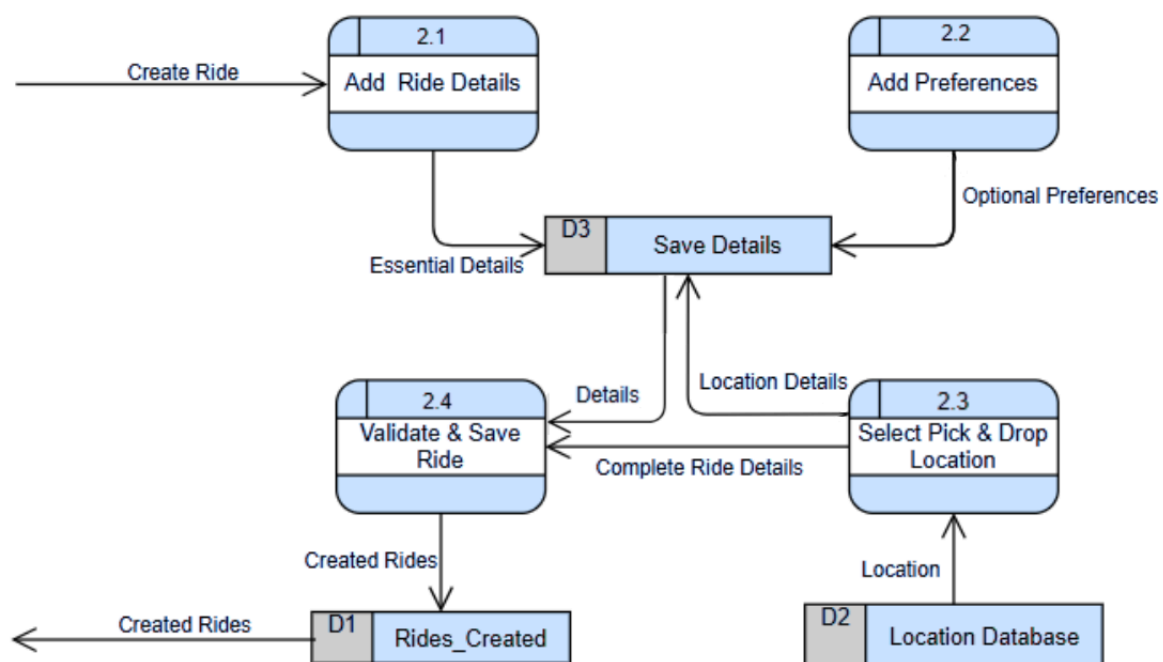
1. **Authenticate User** is the first step where the student logs in by providing their credentials, which are verified through the Credential Database to grant access.
2. **Create Ride** allows a student to post a ride after necessary details such as destination, time, and number of available seats are provided. A new record is created in the Rides_Created data store so the ride can be visible to other users.
3. **Join Ride** lets students browse and choose available rides to join, and the information is then saved in the Rides_Joined data store for tracking.
4. **Confirm Ride** receives and updates statuses for the selected ride using information from both created and joined rides, of which the student is also notified.
5. **Initiate Payment** processes the transaction by sending payment information to the Payment Client and receives confirmation on whether the payment was successful.
6. **Communicate with External Services** connects the system to third-party ride-booking services (like Uber) by sending a ride request and receiving relevant confirmations or additional details.



Child Diagram for “Create a Ride” process

The Child Diagram for Process 2, Create Ride, decomposes the ride creation workflow into four subprocesses, showing how data flows across each step and through supporting data stores. It provides a detailed view of how a student initiates and completes the process of ride creation.

1. The **Add Ride Details** process begins when the student inputs core ride information such as the date, time, available seats, and ride type. These details are stored temporarily in the Save Details data store, allowing them to be accessed and modified during the ride creation process.
2. **Add Preferences** allows the student to provide personal preferences, such as preferred co-rider gender or department. These personalized filters are also saved in the Save Details data store, for a more comfortable and customisable ride-sharing experience.
3. **Select Pick & Drop Location** handles the student's choice of pickup and drop-off points. The process fetches available location options from the Location Database data store, the selected locations are stored in the Save Details data store.
4. Lastly, the process **Validate & Save Ride** retrieves all compiled information from the Save Details data store, checks for completeness and correctness, and saves the ride into the Rides_Created data store so it is now visible to other users.



2. Refinement and Physical DFD Considerations

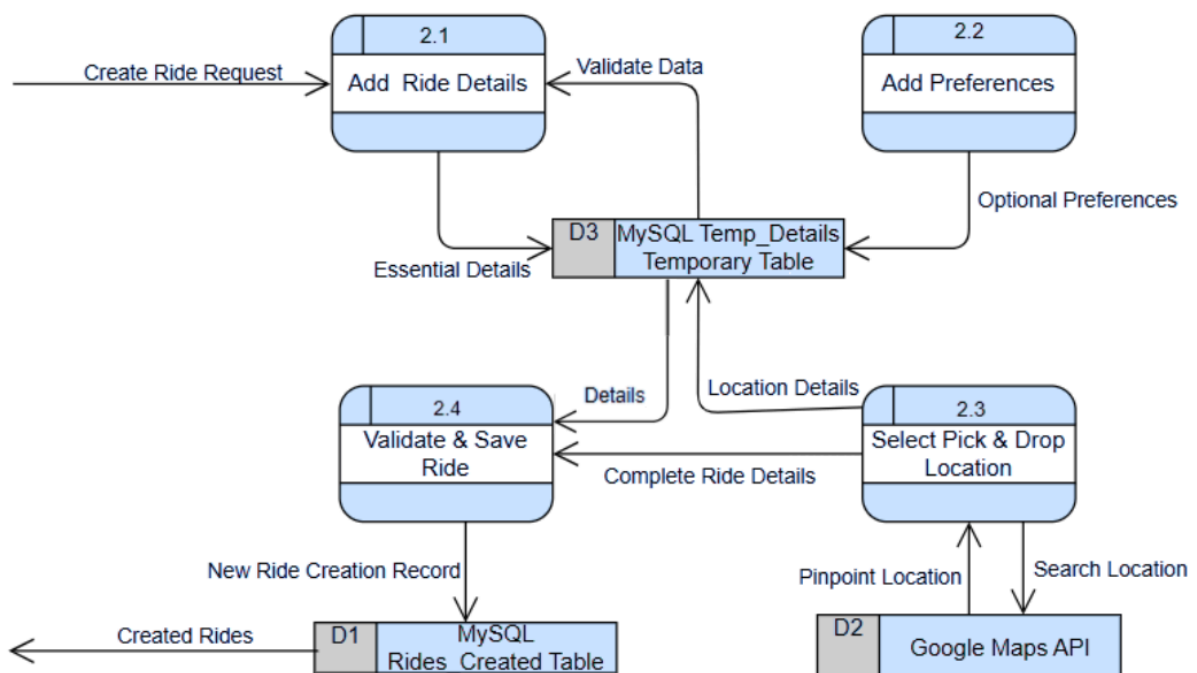
Error-Checking Summary (DFD Rule Violations)

- Violation: Vague data store names used (e.g., "Save Details").
Correction: Renamed data stores like “Temp_Details” for clarity and consistency.
- Violation: External entity (Google Maps API) not represented for map/location data.
Correction: Added external entity “Google Maps API” to reflect proper interaction – user searches for general area and receives a pinpoint location from the API.

Partial Physical DFD

The following diagram shows how a student’s ride request is processed:

1. 2.1 Add Ride Details: Collects core input (e.g., time, seats).
2. 2.2 Add Preferences: Captures optional inputs (e.g., department, gender).
3. 2.3 Validate & Save Ride: Combines and stores in permanent table Rides_Created.
4. 2.4 Select Pick & Drop Location: Integrates Google Maps API to select locations.
5. D3 MySQL Temp_Details Table: Stores intermediate inputs for temporary use, preserving data since the form is split into multiple pages.
6. D1 MySQL Rides_Created Table: Stores the information of created rides.



Naming Conventions:

- Processes: Verb phrases have been used (e.g., "Add Ride Details")
- Data Stores: Prefixed with database type (e.g., "MySQL Temp_Details Table")
- External Entities: Clearly labeled (e.g., "Google Maps API")
- Data Flows: All are labeled with descriptive nouns (e.g., "Location Details", "Optional Preferences")

Control Annotations:

- Create Ride Request → 2.1: Initiates ride creation by the user.
- 2.1 → D3 (Temp_Details): Saves essential ride info temporarily.
- 2.2 → D3: Stores optional preferences such as co-rider gender.
- 2.3 → D2 (Google Maps API): Sends location search request.
- D2 → 2.3: Receives pinpointed location from API.
- 2.3 → D3: Stores selected pick-up/drop-off points.
- D3 → 2.4: Sends complete ride info for validation.
- 2.4 → D1 (Rides_Created): Saves new ride to main database.
- D1 → Output: Confirms ride creation to the user.

CRUD Matrix

The CRUD matrix plays a crucial role in system design by clearly defining which processes have permission to create, read, update, or delete data within each data store. This ensures proper access control by enforcing role-based permissions during development. Additionally, it clarifies the responsibilities of each process, for example, only the "Create Ride" process can generate a new ride listing, while only the "Join Ride" process can initiate a ride request. The matrix also supports data normalization and validation by allowing developers to cross-reference it with data dictionaries, ensuring that unnecessary updates or deletions are avoided, such as preventing direct deletion of ride requests. Lastly, the CRUD matrix aids in system maintenance and planning by helping developers identify which processes to inspect or test when changes are made to the structure or rules of a particular data store.

Process	User Data (Profile, Verification)	Ride Listings (Created Rides)	Ride Requests (Join Requests & Status)
1. Authenticate User	C, R, U, D	-	-
2. Create Ride	R	C	-
3. Join Ride	R	R	C
4. Confirm Ride	R	U	U
5. Initiate Payment	R	R	R
6. Communicate with External Services	R	R	R

3. Partitioning of Physical DFD

Partitioning Analysis:

Two key factors that justify partitioning the physical DFD:

1. Different Execution Environments:

- Processes 2.1 (Add Ride Details), 2.2 (Add Preferences), and 2.3 (Select Pick & Drop Location) are user-facing operations, running on a front-end interface.
- Processes 2.4 (Validate & Save Ride) and 3 (Save Details) involve backend validation and data persistence, handled on the server side.

2. Security and Data Integrity:

- Validation and saving operations require trusted server-side processing to prevent tampering or incorrect ride creation. Thus, processes handling sensitive operations are placed in a secured backend partition.

Partitioned Physical DFD Sketch:

Partition A: Ride Input Interface

Process:

- 2.1 Add Ride Details
- 2.2 Add Preferences
- 2.3 Select Pick & Drop Location

This partition handles user input related to ride creation. It collects data (pickup, drop-off, seats) and filters (like gender, department).

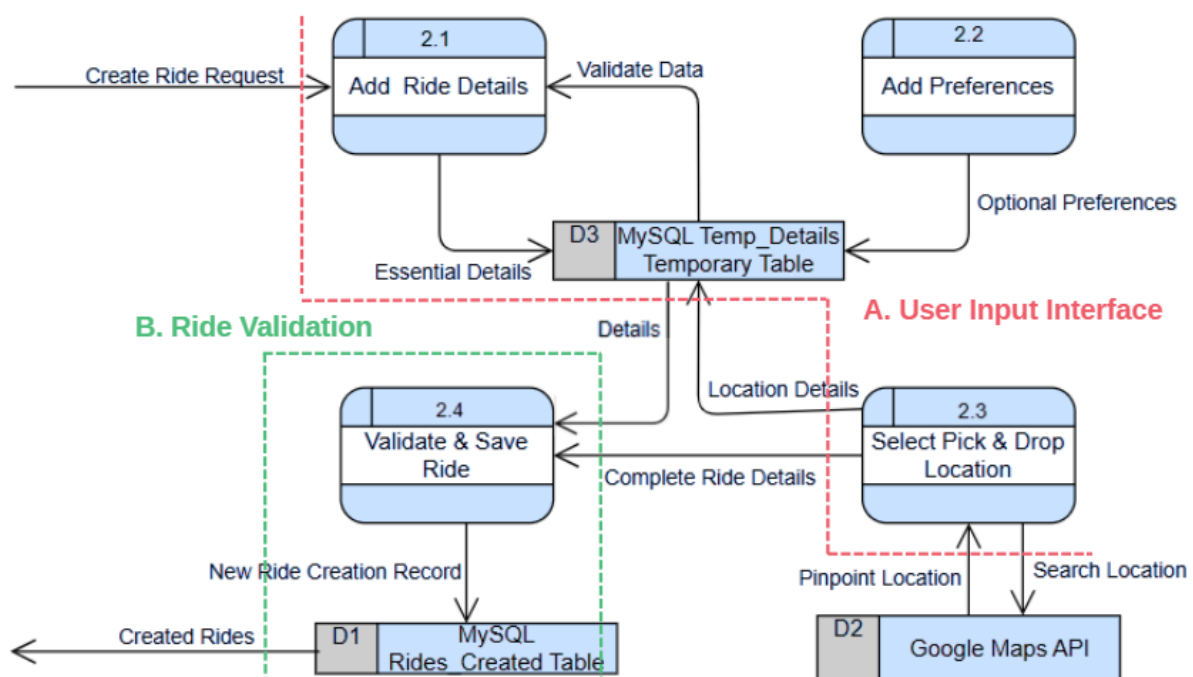
Partition B: Ride Processing & Validation

Process:

- 2.4 Validate & Save Ride

Final processing and database update. This partition ensures the ride data is complete and valid before saving it into the system permanently.

These partitions are clearly delineated with dashed lines in the diagram, following textbook conventions.



Partition Justification:

The two partitions align with both technical requirements and principles.

- Partition A (User Input Interface) is designed for high interactivity and minimal processing. It directly interfaces with the user, allowing quick form inputs and real-time feedback without direct access to databases. This design improves responsiveness and user experience.

- Partition B (Ride Validation) is critical for system integrity and accuracy. Validation, control, and data handling should be done server-side to ensure correctness and prevent malicious input. This partition is where interaction with the permanent database occurs to maintain business logic (e.g., ride completeness, available rides visibility).

Together, these partitions provide a secure, efficient, and user-friendly system that separates concerns effectively, a core recommendation in system design methodologies.